

LOGIN PAGE

The page will redirect user to Keycloak login page, if user is not logged in (*Figure 5.5*). The UI is equipped with feature such as hide password, and register at the bottom of the screen.

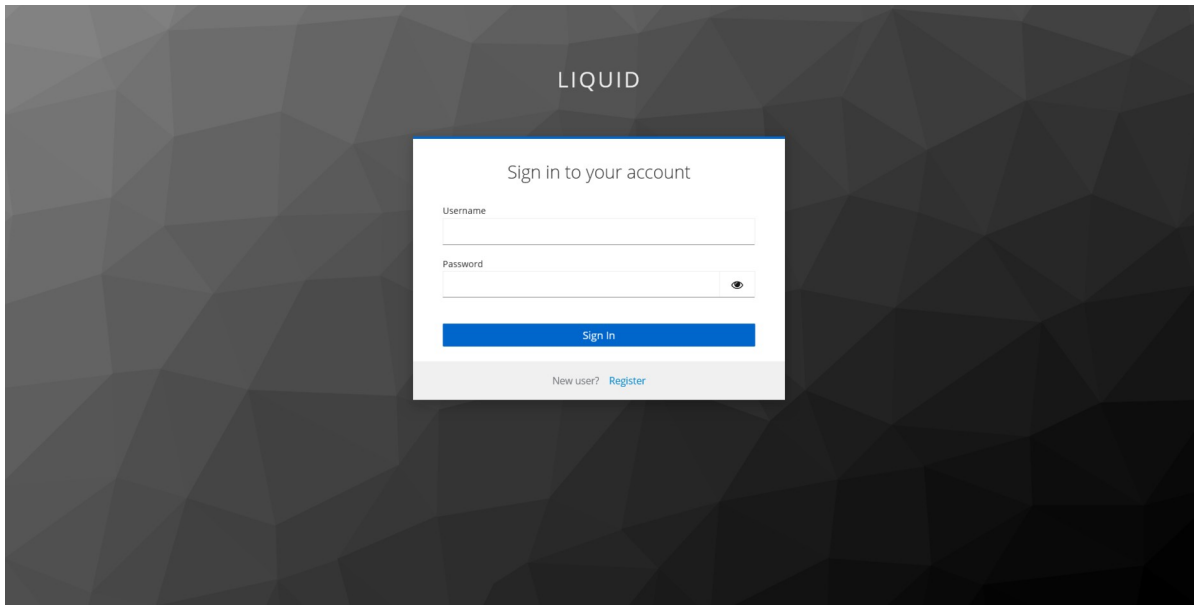


Figure 5.5

REGISTER PAGE

The register page (*Figure 5.6*) will show up when the user clicked the register text. The register page annotates the required field with * (star) sign, the emails is optional (left out). User may click “Back to login” if necessary. Validation is done client-side and server-side.

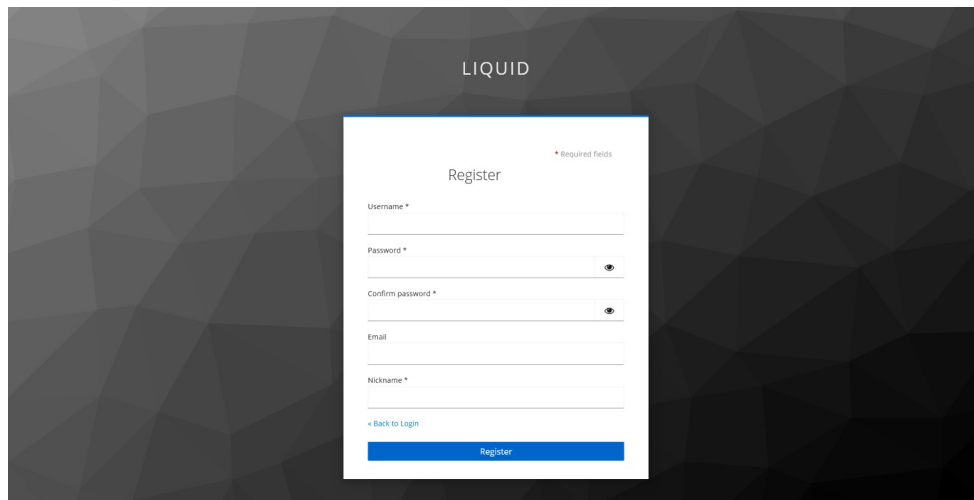
A screenshot of a web application's registration page. The page has a dark, geometric, low-poly background. In the center, there is a white rectangular form titled "Register". Above the form, the word "LIQUID" is displayed in a light gray font. The form contains several input fields: "Username *" (required), "Password *" (required, with a toggle for visibility), "Confirm password *" (required, with a toggle for visibility), "Email" (optional), and "Nickname *" (required). Below the input fields, there is a link that says "« Back to Login" and a blue button labeled "Register". A small red asterisk and the text "* Required fields" are located in the top right corner of the form area.

Figure 5.6

REGISTER PAGE (VALIDATE CLIENT-SIDE)

When triggered, a notice below the input will inform the user by displaying a red-colored text and says specifically which error they have and how can they be fixed (*Figure 5.7*).

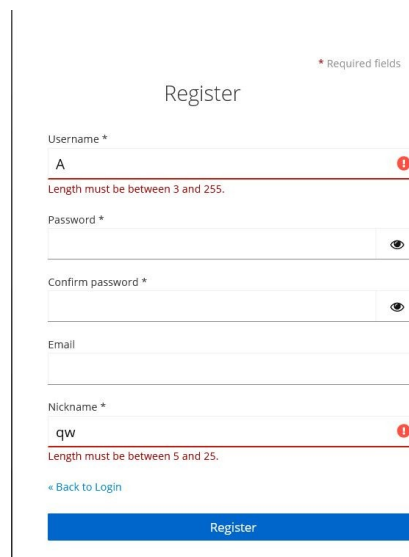
A screenshot of the same registration page as in Figure 5.6, but with client-side validation errors. The "Username *" field contains the text "A" and has a red error message below it: "Length must be between 3 and 255." The "Nickname *" field contains the text "qw" and has a red error message below it: "Length must be between 5 and 25." Both error messages are preceded by a small red circle containing an exclamation mark. The "Password *" and "Confirm password *" fields are empty. The "Email" field is also empty. The "« Back to Login" link and the "Register" button are still present at the bottom of the form.

Figure 5.7

WELCOME PAGE

The user will be greeted by an alert pop-up to make private password for their private keys (*Figure 5.8*).

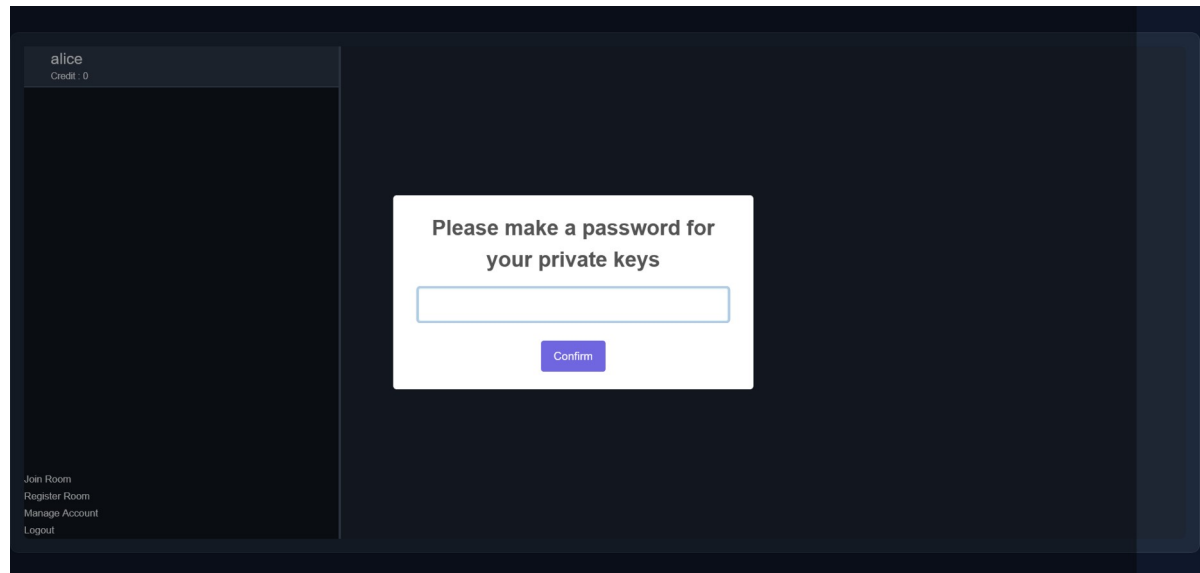


Figure 5.8

And the welcome page (*Figure 5.9*). on the top corner lies the nickname's user and credit they own, the bottom side has Join, and register room, and manage, aswell as logout.

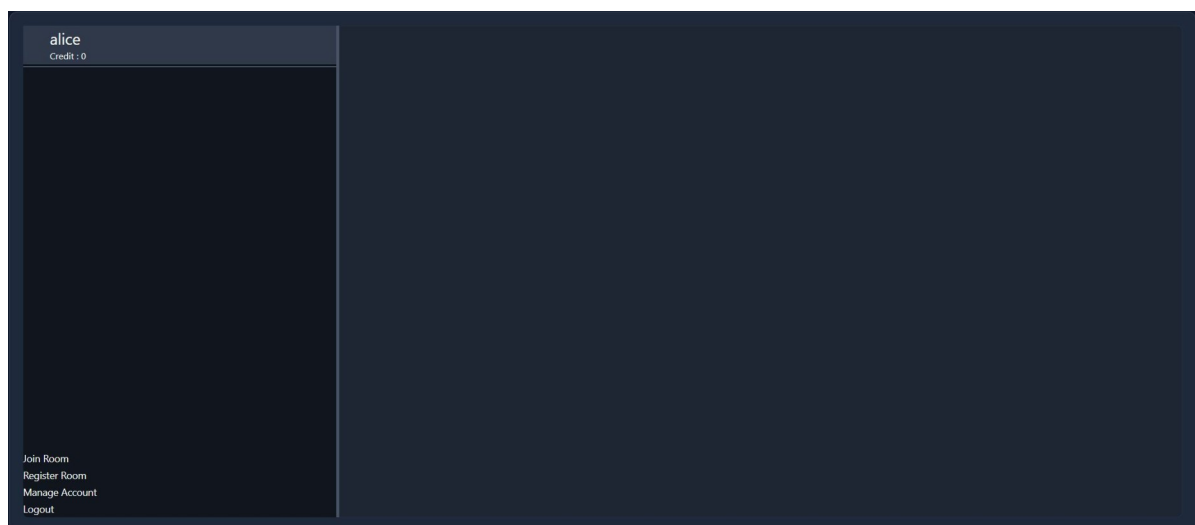


Figure 5.9

ACCOUNT MANAGEMENT (PERSONAL INFO)

User are redirected to Keycloak’s user account management, when clicking “Account management” (Figure 5.10). Here, user can edit their nickname, the credit and username are immutable. User may click “Back to backend-service” when they are satisfied editing their account attribute.

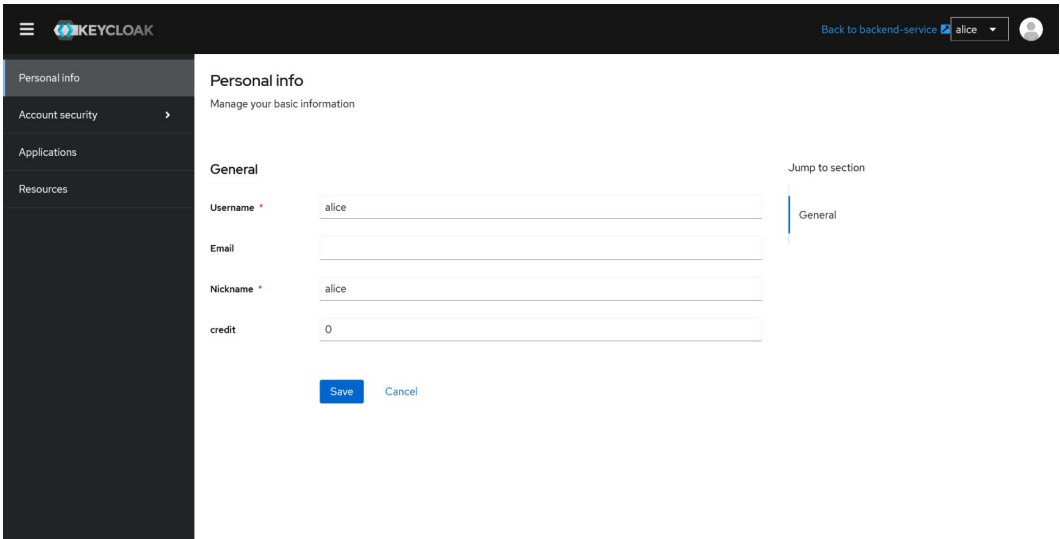


Figure 5.10

ACCOUNT MANAGEMENT (SIGNING IN)

On visiting “Signing in” tab, user may add 2FA to further enhance security, and change password whenever the case. (Figure 5.11)

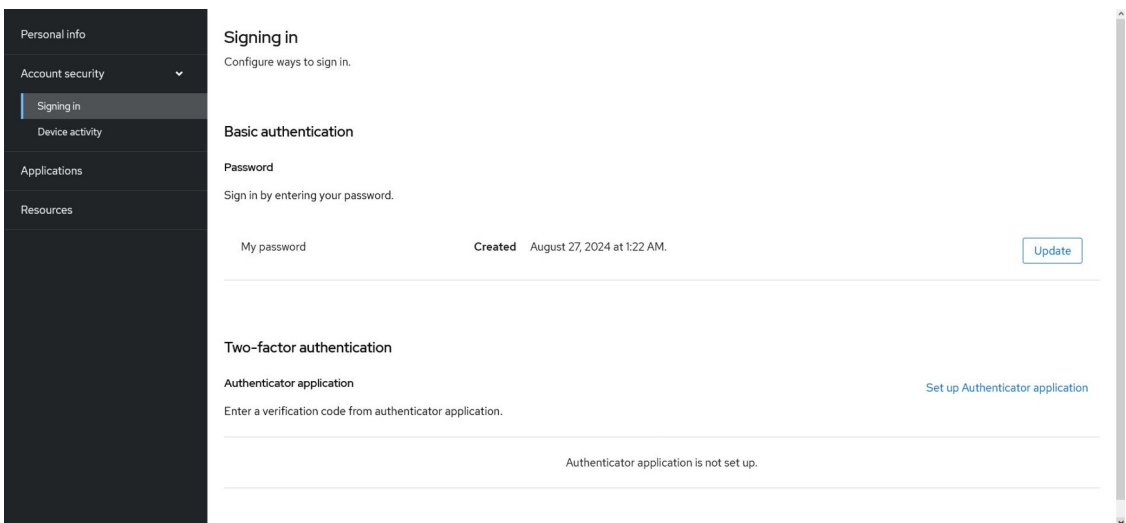


Figure 5.11

And you can monitor your signed in presence if you visit the “Device Activity” tab. (Figure 5.12)

Device activity

Sign out of any unfamiliar devices.

Signed in devices

[Refresh the page](#)

Windows 10 / Firefox/131.0

Current session

IP address

172.29.0.1

Started

August 27, 2024 at 1:22 AM

Last accessed

August 27, 2024 at 1:31 AM

Expires

August 27, 2024 at 11:22 AM

Clients

backend-service, \${client_account-console}

Figure 5.12

HOME PAGE (REGISTER ROOM)

If a user wants to chat with other user, they need to create a room first. In order to create a room, they can hover their mouse on “Register room” and click it, a prompt saying “Are you sure” will appear (*Figure 5.13*) and check if they want to register, if “Yes”, a room service API will be called to make a new room with the first participant as the user, over on the Mongo DB side.

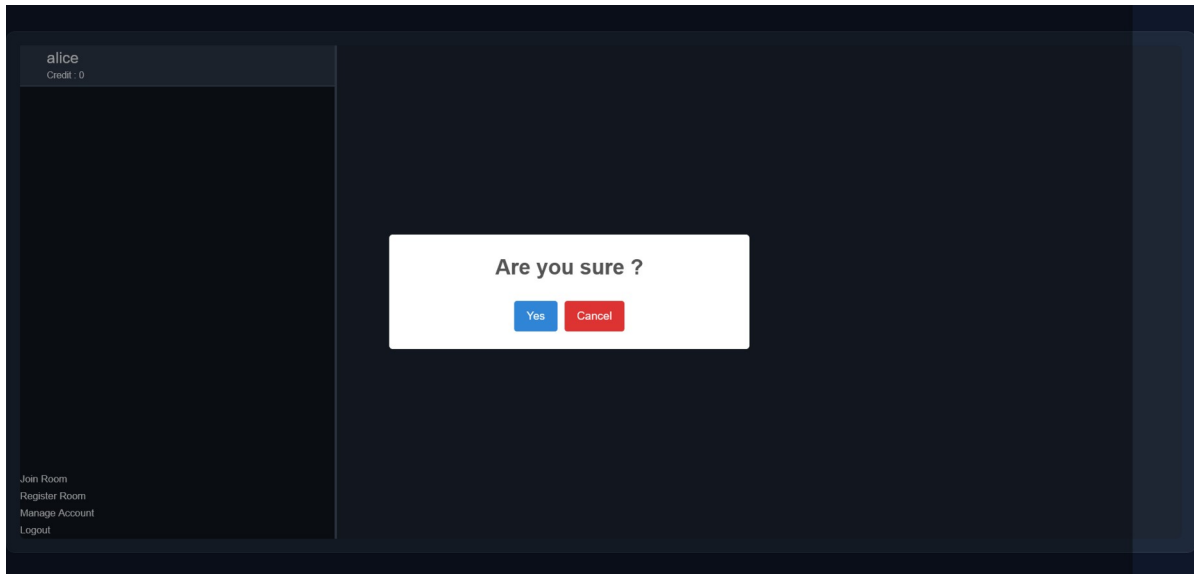


Figure 5.13

And as a result, a div is rendered on the client-side and will show one added tab, where a user can join into the new room they just created. (*Figure 5.14*)

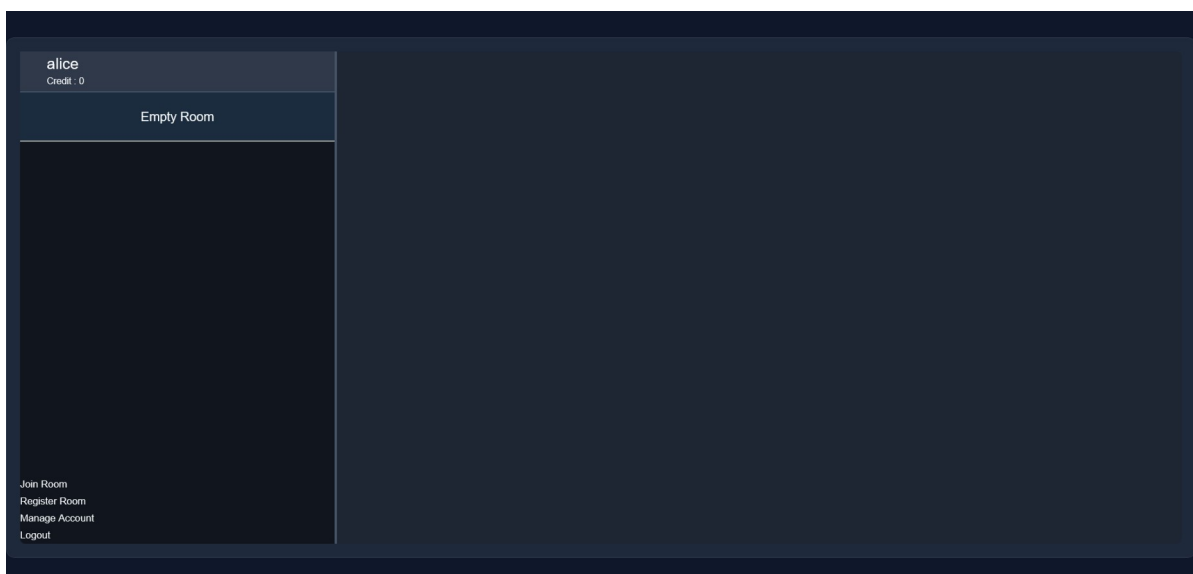


Figure 5.14

HOME PAGE (JOIN ROOM)

In order to demonstrate another participant, I created another new account and join the room alice just made. When “Join room” is clicked, an input form will emerge (*Figure 5.15*). The input requires the room ID (URL Bar). A room needs to exist in order for it to work.

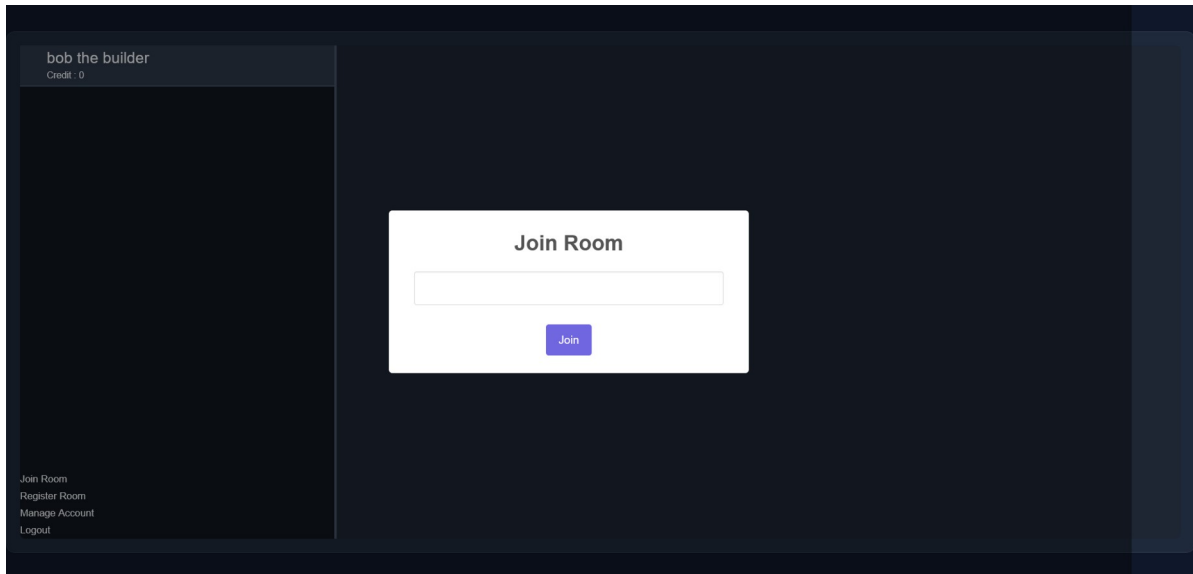


Figure 5.15

When an input is successful, and another person is joined in, the title would change from “Empty Room” To the other participants name. (*Figure 5.16*)

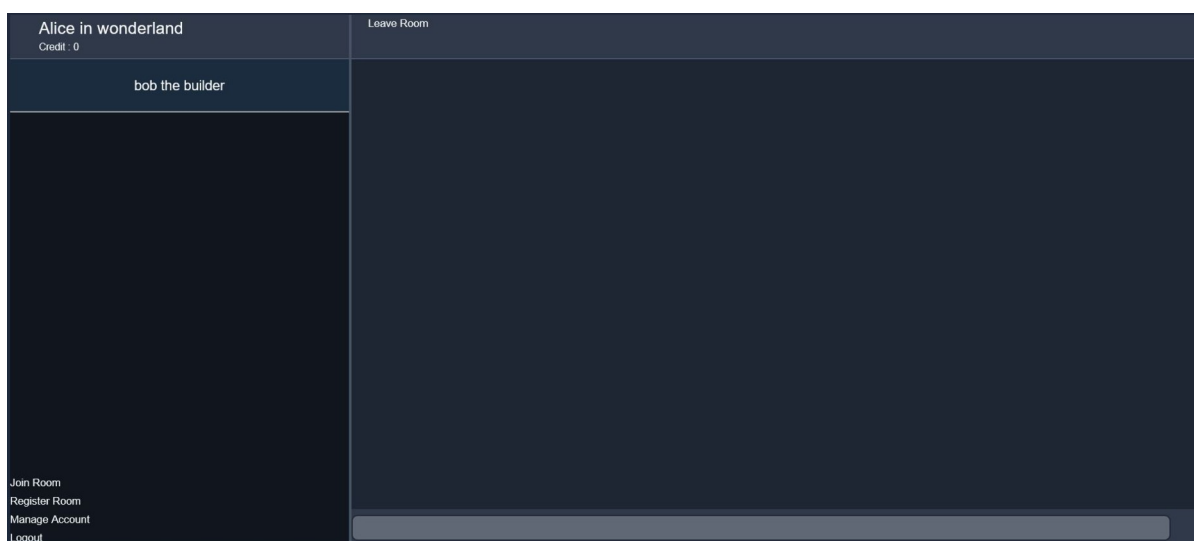


Figure 5.16

SENDING A MESSAGE (NOT ENOUGH CREDIT)

When a user has 0 credit, like Alice here, a popup will appear and ask the user to top-up their credit, as for now a dummy API is created to simulate a topup. (*Figure 5.17*)

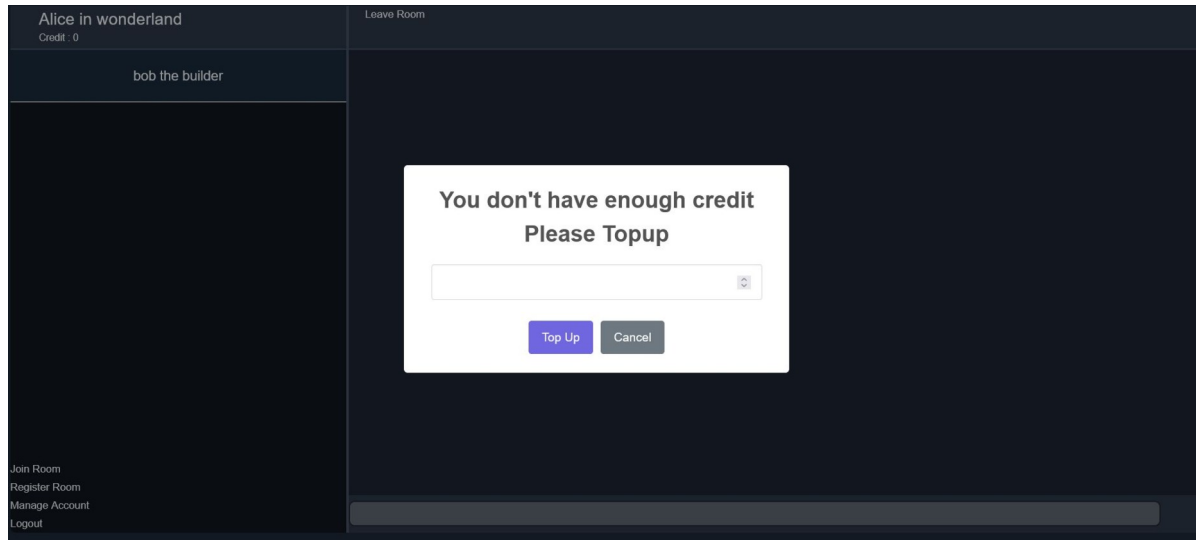


Figure 5.17

And when success, the top bar credit status will change reactively. (*Figure 5.18*)

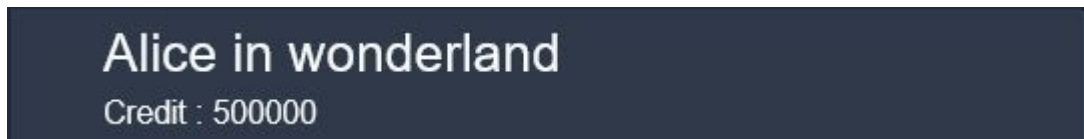


Figure 5.18

SEND MESSAGE (ENCRYPTED)

When sending a message, an encrypted text will appear in text chat. These text are shown purposely to demonstrate the end-to-end encrypted flow in the chat. (Figure 5.19)

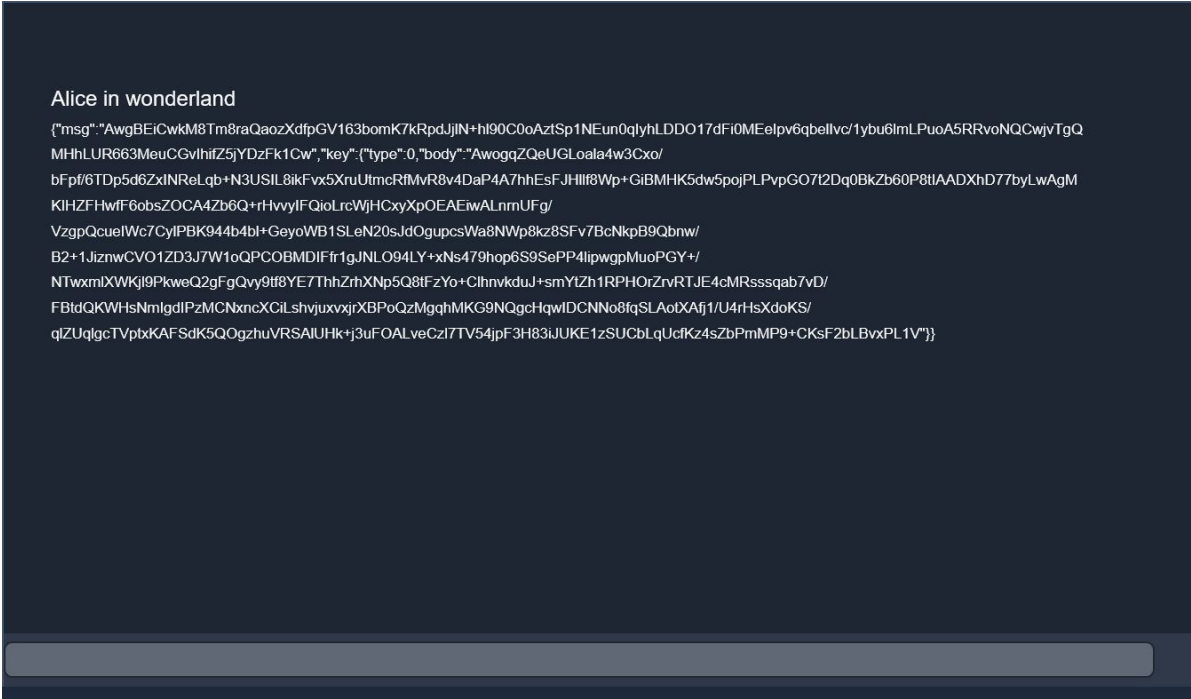


Figure 5.19

And the decrypted message is shown in Figure 5.20

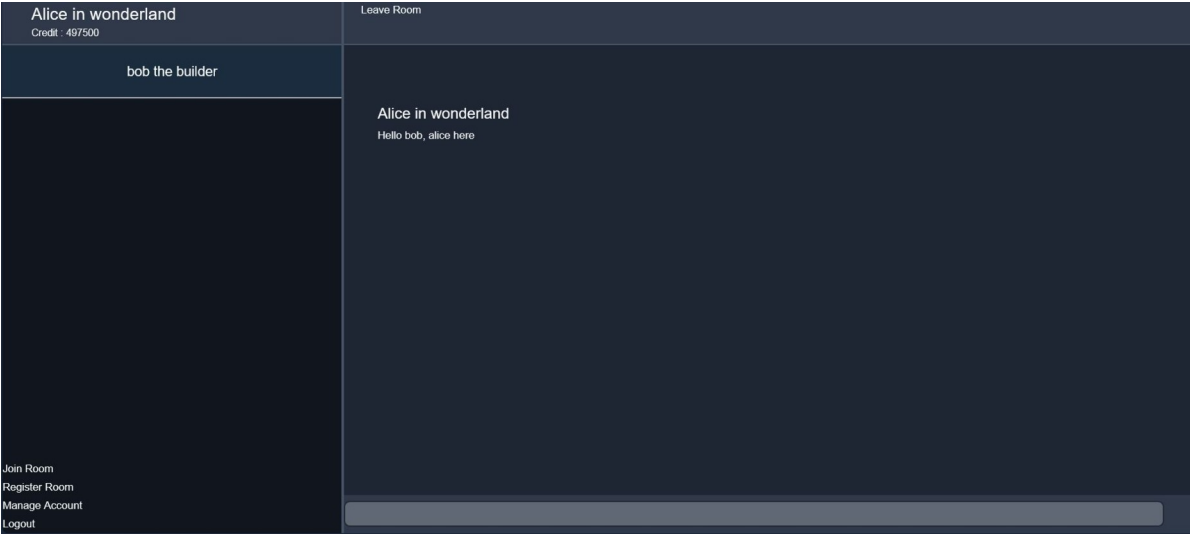


Figure 5.20

SEND MESSAGE (FROM BOB'S POV)

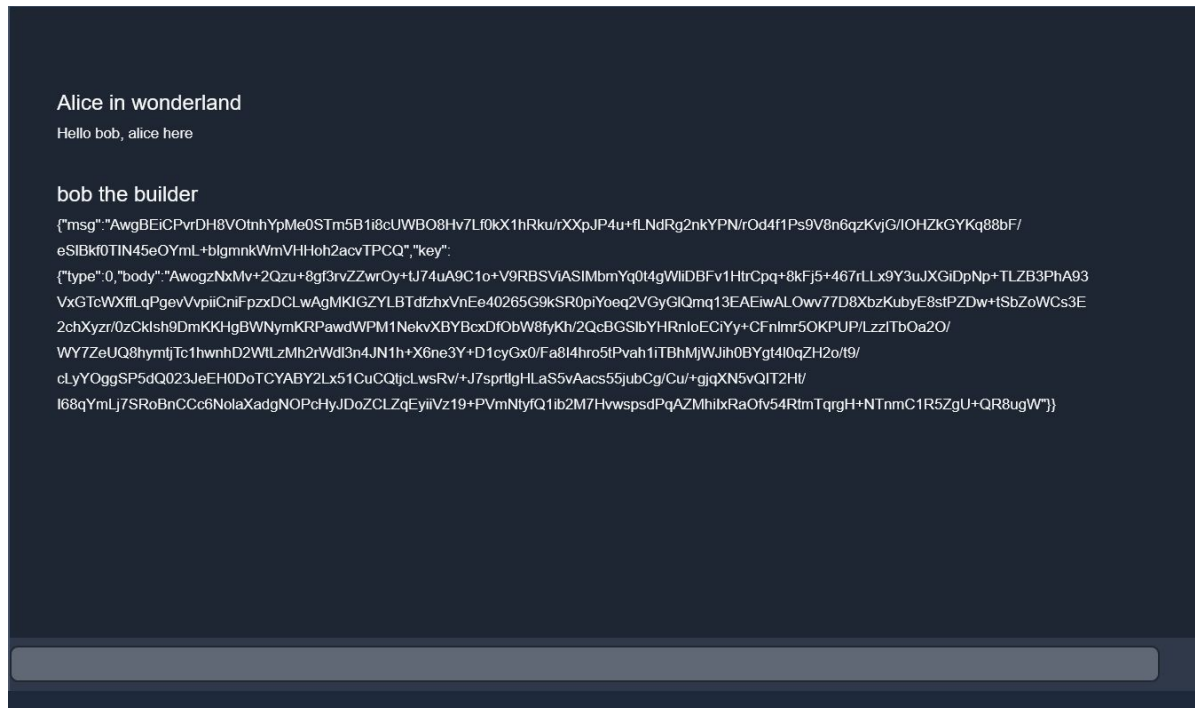


Figure 5.21

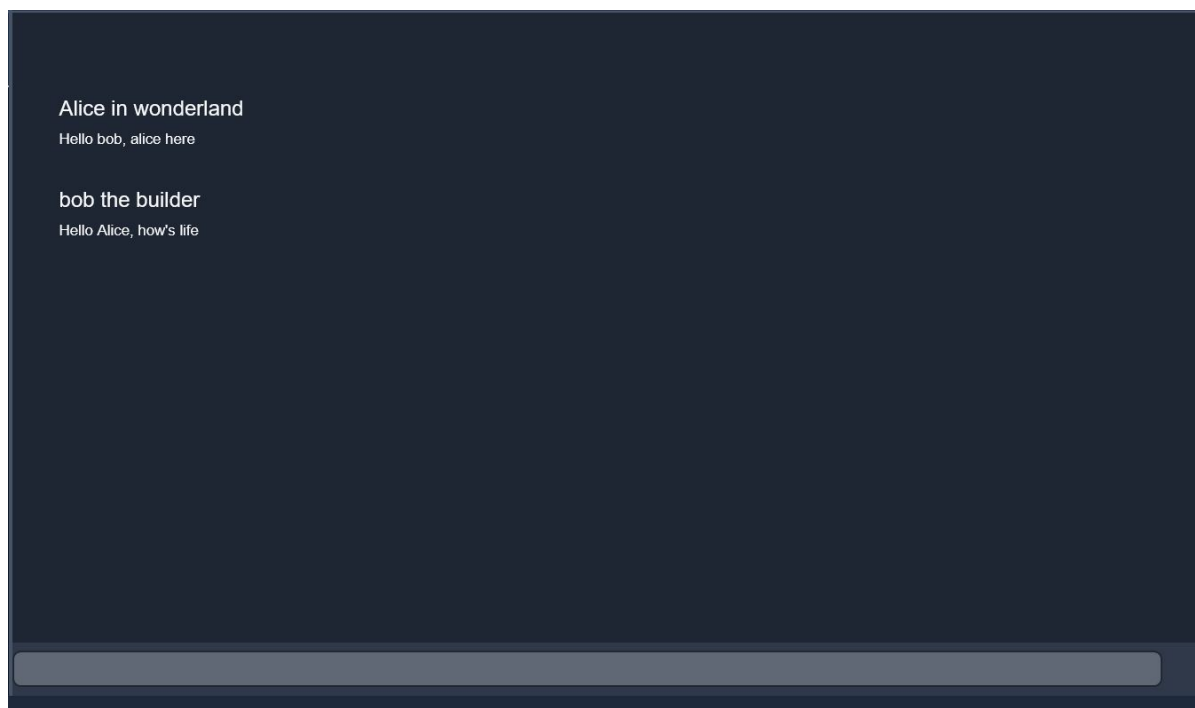


Figure 5.22

REDIS (PUB/SUB)

Whenever a user sends a messages a log can be seen over at “Tiny RDM” Application. *Figure 5.23* shows that the encryption works; the server cannot decrypt the messages, and doesn’t know what the participants are talking.

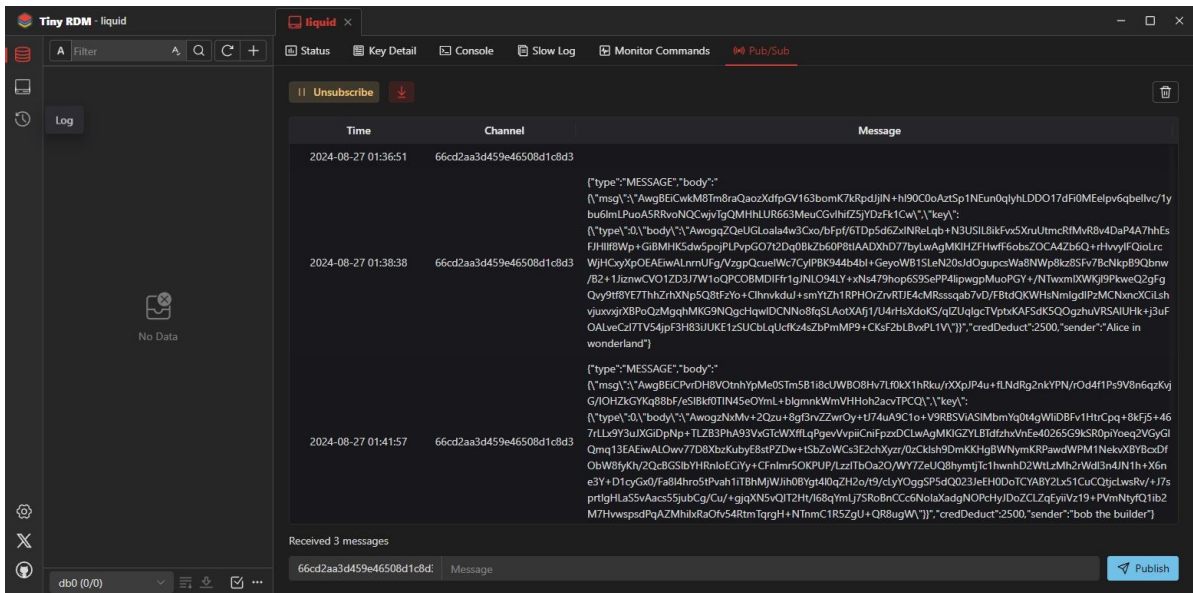


Figure 5.23

KAFKA

When a user is registered (REGISTER type) / changed (UPDATE_PROFILE) their profile attribute, an event is called on “liquid-keycloak-event” topic (*Figure 5.24*) ,

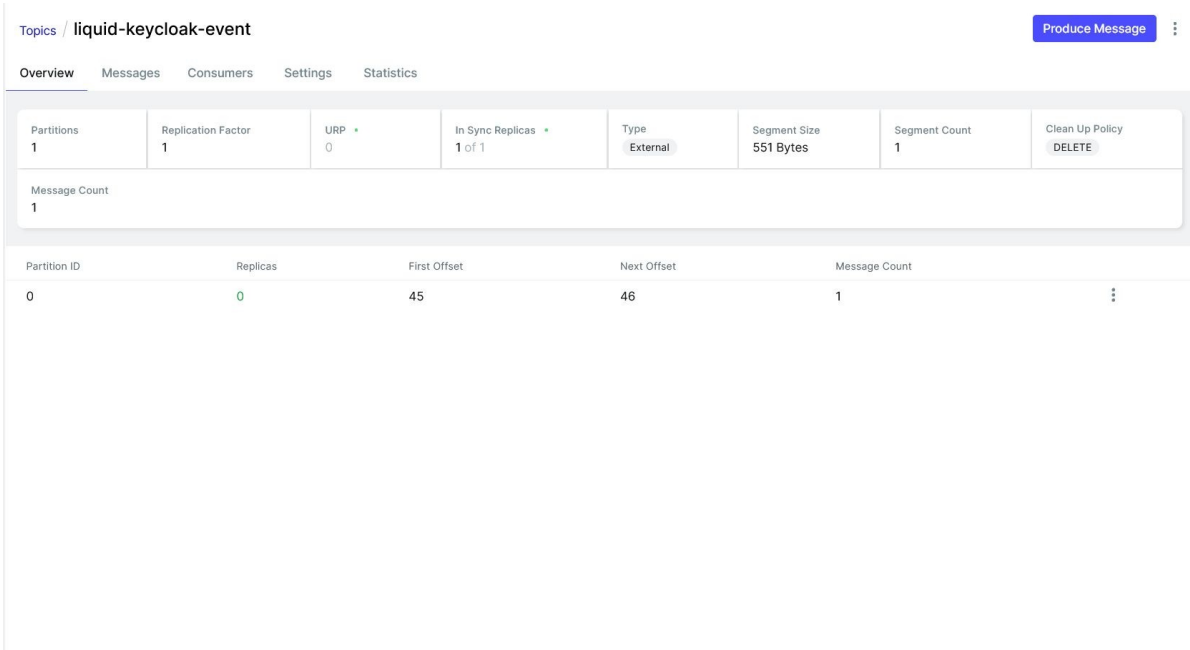


Figure 5.24

In this case, “liquid” (Liquid IM) are the consumer. (*Figure 5.25*)

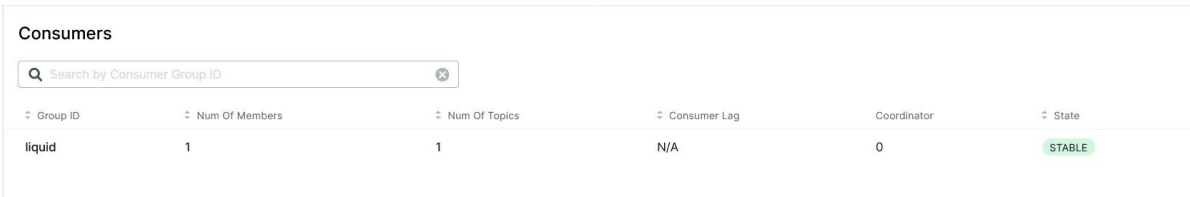


Figure 5.25

Figure 5.26 shown what keycloak would send to kafka whenever a user decides to register. This information then get received by liquid, and used as a way to synchronized database.

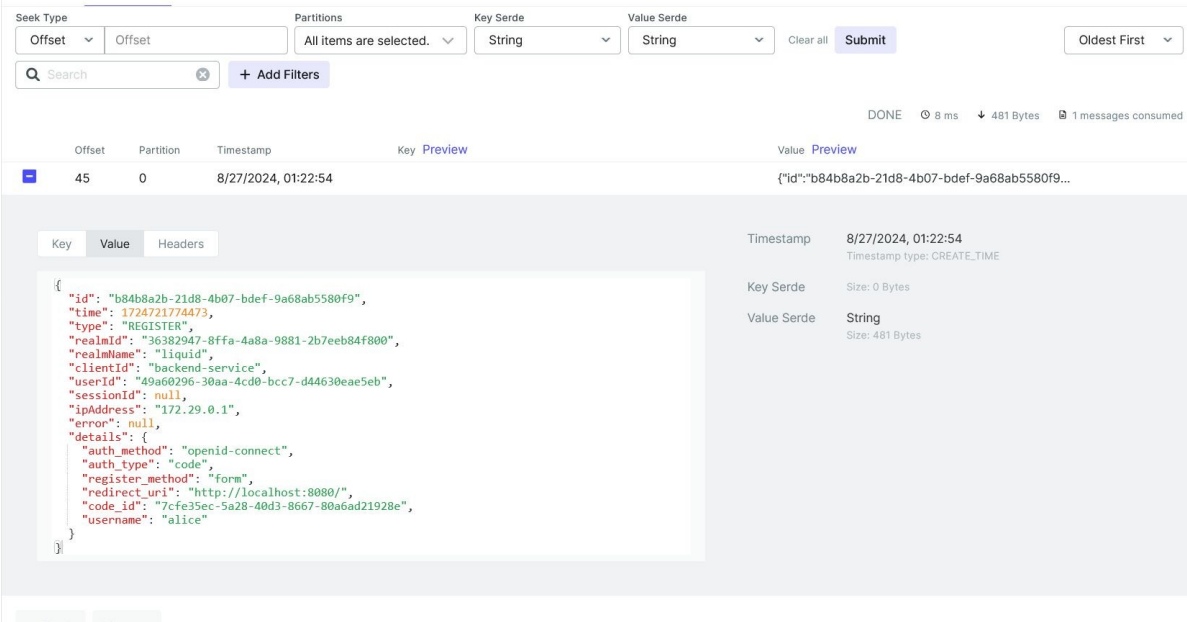


Figure 5.26

And this is what it would look like for “UPDATE_PROFILE” event. (Figure 5.27)

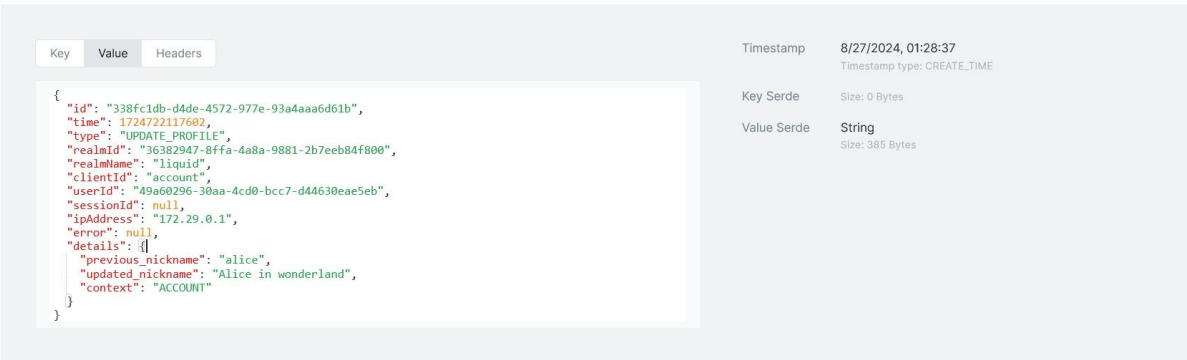


Figure 5.27

MONGO (MESSAGES)

Messages are stored in Mongo DB due to performance issue in postgreDB (Figure 5.28)

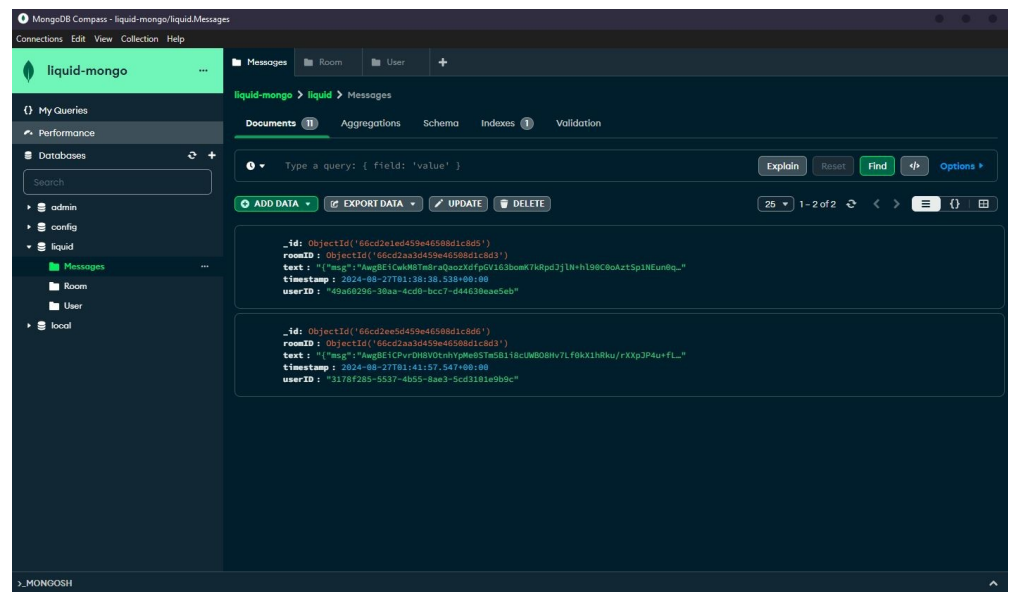


Figure 5.28

MONGO (USER)

A user is automatically synchronized (created) when an event from kafka (register) is received on the backend (Figure 5.29) . A user database is then also synchronized when a user updates their profile. For example, in Figure 5.30, Alice updates her nickname to “Alice in wonderland” as a result, the database is updated.

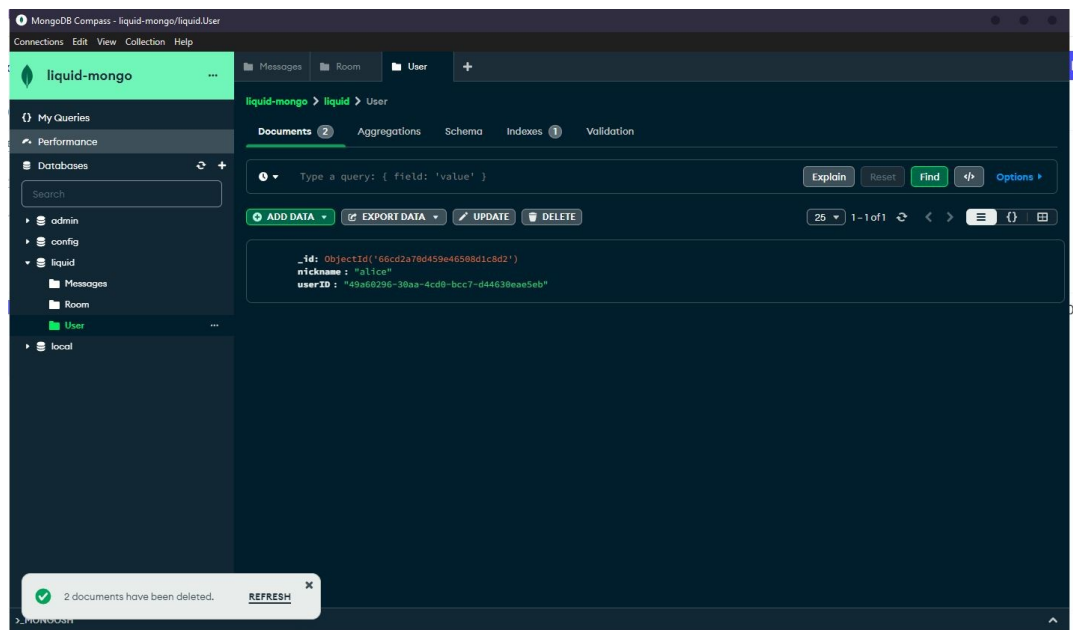


Figure 5.29

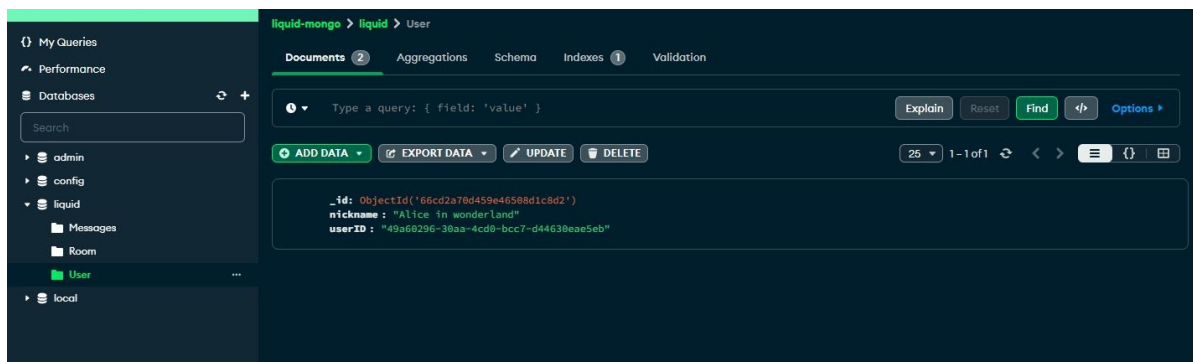


Figure 5.30

MONGO (ROOM)

Room database is also stored in mongoDB due to high concurrent. A room is filled with “participants” (array), which is IDKeys (Identity Keys), OTKeys (One-time keys), and userID (profile ID). (Figure 5.31)

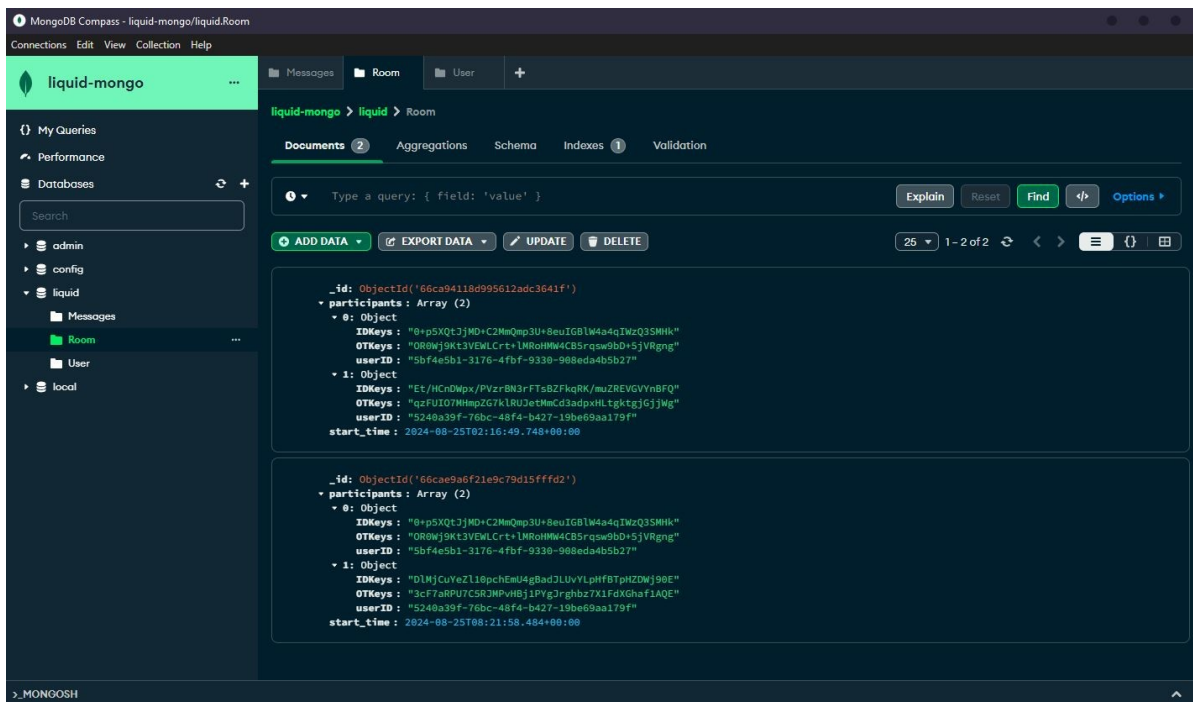


Figure 5.31

POSTGRES SQL

Postgre DB holds the user permanent, and semi state attrbiute, such as, user_attribute (keycloak) table contains the user’s attribute, where it contains the static nickname (Read only), and credit amount. (Figure 5.32)

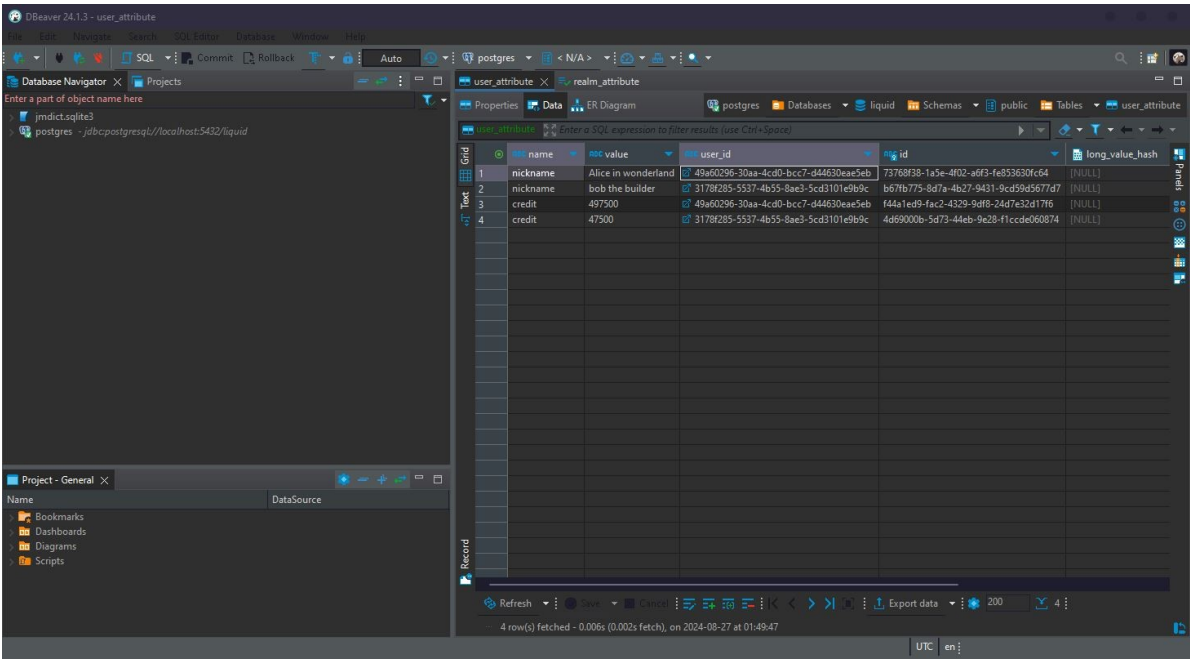


Figure 5.32